

DOCUMENTO DE ANÁLISE CRÍTICA: PROJETO AANC

Disciplina: AI Factory: Building Intelligent Systems

Instituição: Pontifícia Universidade Católica do Paraná (PUCPR)

Projeto: AANC - Agente de Apoio à Negociação



PARTE A: REVISITANDO O DESAFIO

1. Link de Acesso ao Sistema (Deploy)

- **URL Pública:** <https://aanc---gestor-de-negocia-es-8rs8bvoyk4fensya7muovq.streamlit.app/>

2. Alinhamento com a Metodologia CBL (Challenge-Based Learning)

- **Grande Ideia:** Inteligência Artificial Aplicada às Relações Trabalhistas e Sindicais.
- **Pergunta Essencial:** Como a inteligência artificial generativa e preditiva pode auxiliar gestores de People Analytics e relações sindicais a tomarem decisões rápidas, seguras e financeiramente sustentáveis durante negociações coletivas?
- **Desafio:** Desenvolver um agente inteligente e seguro (AANC) que unifique dados salariais (via consultas SQL determinísticas), previsões de retenção/turnover (via modelos preditivos de Machine Learning) e análise de políticas sindicais (via RAG sobre Acordos Coletivos de Trabalho - ACT/CCT).

3. Avaliação de Resolução do Desafio: Em que medida o sistema resolve o problema?

O sistema desenvolvido resolve o desafio proposto em **grande medida no plano arquitetural**, mas de forma **parcial no ambiente de produção na nuvem**.

- **No plano local (Desenvolvimento):** O AANC provou ser extremamente eficaz. Ele conseguiu unificar com sucesso a precisão de cálculos determinísticos (DuckDB) à inteligência de agentes autônomos de decisão, eliminando o risco de alucinação matemática em relatórios de folha de pagamento. Além disso, o acoplamento do modelo preditivo de Machine Learning permitiu simulações de risco de turnover em tempo real de acordo com as propostas de reajuste.
- **No plano público (Produção):** Embora a arquitetura esteja consolidada e o deploy tenha sido concluído com sucesso no Streamlit Community Cloud, limitações severas de infraestrutura gratuita impedem que o sistema opere

com 100% de estabilidade de forma contínua, gerando falhas de runtime e conflitos de dependências que limitam sua usabilidade imediata pelo usuário final.

PARTE B: ANÁLISE TÉCNICA DO SISTEMA

Uma avaliação técnica realista e aprofundada aponta forças estruturais significativas, mas também gargalos severos de integração e infraestrutura.

1. O que funciona bem no sistema e por quê?

- **Motor de Cálculo Determinístico (SQL via DuckDB):** A segregação de tarefas de cálculo para um banco de dados relacional funcionou de forma impecável. O modelo de linguagem (Gemini) gera as queries SQL de forma limpa, e o DuckDB executa os cálculos de impacto financeiro (incluindo reflexos de férias, 13º e encargos sociais) com precisão absoluta, garantindo a confiança contábil necessária para a tomada de decisão em RH.
- **Simulação Preditiva de Turnover (Machine Learning):** A integração do pipeline treinado com Scikit-Learn (ml/modelo_turnover.pkl) funciona de maneira ágil. O modelo recebe os inputs de reajuste salarial e recalcula instantaneamente as probabilidades de evasão de pessoal (turnover), oferecendo suporte analítico direto a partir do dataset do mundo real (IBM Attrition).
- **Rastreabilidade e Observabilidade (Langfuse):** A instrumentação do código-fonte para enviar os spans e o rastreamento das chamadas da LLM e da CrewAI para o Langfuse está funcional. Isso permite realizar auditorias em tempo real de latência, custos e taxa de acerto do modelo.

2. O que não funciona (ou funciona mal) e por quê? (Análise de Falhas)

- **Falsos Positivos no Pipeline de Segurança (safety_layer.py):** O mecanismo de segurança implementado para evitar ataques de *Prompt Injection* e desvios de escopo operou de forma rígida demais. O filtro de entrada analisa o prompt buscando termos de controle e imperativos corporativos. No entanto, perguntas legítimas do usuário final (ex: "Qual é a regra para férias?") acionavam o gatilho de segurança indevidamente, classificando a busca de políticas de RH como uma ameaça e bloqueando a resposta do modelo com a mensagem padrão: *"Não posso ajudar com esse tipo de solicitação..."*.
- **Gargalo de Memória e "Cold Start" no Streamlit Cloud:** A biblioteca de embeddings txtai (utilizada para o RAG) e as bibliotecas científicas de ML (como o PyTorch/Scikit-Learn implícitos) são pesadas. No tier gratuito do

Streamlit Cloud, que disponibiliza apenas 1GB de RAM de forma compartilhada, a inicialização do app do zero (*cold start*) causa estouro de memória (Out-Of-Memory). Isso resulta em erros intermitentes de renderização de tela e travamentos no frontend baseados em React (como o erro de nó DOM corrompido `removeChild`).

- **Limitação na Recuperação de Contexto do RAG:** O fatiamento estático de 1000 caracteres dos PDFs de acordos coletivos (*chunking linear*) feito no `rag_engine.py` fragmentou as tabelas de dados e cláusulas contíguas. Quando o usuário busca por termos genéricos, o motor vetorial "erra a mira" e traz blocos semânticos incorretos (ex: puxar a cláusula de contribuição sindical ao invés da regra de férias). Como o prompt de sistema proíbe alucinações, o modelo se recusa a responder, gerando frustração na experiência de uso.

3. Limitações Conhecidas

- **Limitação de Escopo:** O RAG está restrito a documentos em formato PDF estático e não faz o cruzamento de informações entre múltiplos arquivos de plantas diferentes de forma comparativa automática.
- **Limitação de Hardware:** Incapacidade de rodar modelos locais de embeddings mais precisos devido às restrições de CPU/RAM da hospedagem em nuvem gratuita.

PARTE C: VISÃO DE FUTURO

Para transformar o protótipo acadêmico atual em um produto SaaS robusto de nível empresarial para People Analytics, propõem-se três evoluções tecnológicas concretas:

Evolução 1: Migração de Infraestrutura para Containers Dedicados

- **O que mudaria:** O sistema deixaria de ser hospedado no Streamlit Cloud compartilhado e passaria a rodar em uma infraestrutura containerizada com autoescalonamento.
- **Tecnologia Utilizada:** Docker, Kubernetes e hospedagem via AWS ECS (Elastic Container Service) ou Google Cloud Run, com teto de memória dinâmico.
- **Impacto Esperado:** Eliminação completa de erros de falta de memória (OOM), redução drástica do tempo de *cold start* de minutos para segundos e estabilização de requisições paralelas concorrentes.

Evolução 2: Refatoração do RAG com Chunking Semântico e Parent-Document Retrieval

- **O que mudaria:** Substituição do fatiamento estático por um fatiamento inteligente que respeite a estrutura lógica de parágrafos, cláusulas e tabelas do Acordo Coletivo.
- **Tecnologia Utilizada:** LlamaIndex, LLM Sherpa (para parsing estruturado de tabelas em PDFs) e armazenamento vetorial no Pinecone ou pgvector (PostgreSQL).
- **Impacto Esperado:** Respostas 100% precisas para consultas de regras de benefícios, férias e jornadas, garantindo que o contexto injetado no Gemini seja sempre o trecho correto do PDF.

Evolução 3: Guardrails Inteligentes baseados em Classificadores (NeMo Guardrails)

- **O que mudaria:** O pipeline rígido baseado em expressões e regex do `safety_layer.py` seria substituído por uma camada semântica flexível de controle de diálogo.
- **Tecnologia Utilizada:** NVIDIA NeMo Guardrails ou Llama Guard.
- **Impacto Esperado:** Redução da taxa de falsos positivos de segurança a zero. O sistema passará a discernir de forma inteligente quando uma pergunta sobre "férias" é um fluxo legítimo de RH e quando é um ataque real de desvio de comportamento.